

Encontro 3: Colaboração no GitHub com Pull Requests e Resolução de Conflitos

Sumário

Encontro 3: Colaboração no GitHub com Pull Requests e Resolução de Conflitos.....	1
1. Aprofundando a Colaboração com Git e GitHub.....	2
2. O que é um Pull Request (PR)?.....	2
3. Fluxo de Trabalho com Pull Requests	2
4. Lidando com Conflitos de Merge	4
4.1. Como Resolver um Conflito.....	4
5. Boas Práticas de Colaboração	5
6. Exercícios Práticos.....	5
6.1. Exercício 1: Criando e Mesclando um Pull Request	5
6.2. Exercício 2: Simulando e Resolvendo um Conflito	5
7. Soluções dos exercícios práticos	6
7.1. Exercício 1: Criando e Mesclando um Pull Request	6
7.2. Exercício 2: Simulando e Resolvendo um Conflito	17
8. Dicas para Projetos Reais	28
9. Ferramentas Auxiliares para Resolução.....	28
10. Boas Práticas para Conflitos.....	29
11. Cenários Avançados de Conflitos.....	30

1. Aprofundando a Colaboração com Git e GitHub

No encontro anterior, aprendemos a sincronizar nosso trabalho com repositórios remotos. Agora, vamos mergulhar no fluxo de trabalho colaborativo padrão da indústria, utilizando **Pull Requests**. Este mecanismo é fundamental para garantir a qualidade do código, facilitar a revisão por pares e manter um histórico de alterações limpo e compreensível no projeto.

2. O que é um Pull Request (PR)?

Um Pull Request é uma solicitação para que as alterações de um branch sejam integradas (merged) em outro. É uma forma de propor modificações no projeto principal. Ao abrir um PR, você inicia uma discussão sobre suas alterações, permitindo que outros membros da equipe:

- **Revisem o código:** Analisem as alterações, sugiram melhorias e identifiquem possíveis bugs.
- **Discutam as mudanças:** Façam perguntas e debatam a implementação.
- **Automatizem testes:** Em muitos projetos, a abertura de um PR dispara testes automatizados para garantir que as novas alterações não quebrem o código existente.

Este processo é central para a metodologia de **Integração Contínua (CI)**.

3. Fluxo de Trabalho com Pull Requests



Figura 1: Diagrama do fluxo de trabalho com Pull Requests no GitHub.

A figura 1 mostra as etapas desde a criação do branch local até a integração final, incluindo o processo de revisão de código que é fundamental para a qualidade do projeto.

O fluxo de trabalho típico ao colaborar em um projeto usando Pull Requests é o seguinte:

1. **Criar um Branch:** Nunca trabalhe diretamente no branch `main`. Sempre crie um novo branch para sua funcionalidade ou correção.

```
# Garante que você está no branch principal e atualizado
git checkout main
git pull origin main
# Cria e muda para o novo branch
git checkout -b minha-nova-feature
```

2. **Fazer as Alterações:** Adicione, modifique ou exclua arquivos e faça commits normalmente em seu branch.

```
# ... faça suas alterações no código ...
git add .
git commit -m "Adiciona a nova feature X"
```

3. **Enviar o Branch para o Repositório Remoto:**

```
git push origin minha-nova-feature
```

4. **Abrir o Pull Request:** No site do GitHub, vá para o seu repositório. O GitHub geralmente detecta que você enviou um novo branch e exibe um botão para "Compare & pull request". Clique nele, adicione um título e uma descrição clara das suas alterações e crie o Pull Request.
5. **Revisão e Discussão:** Sua equipe irá revisar o código, deixar comentários e solicitar alterações se necessário. Você pode fazer novos commits em seu branch e enviá-los para atualizar o PR.
6. **Merge:** Após a aprovação, um mantenedor do projeto (ou você mesmo, dependendo das permissões) fará o merge do seu Pull Request, integrando suas alterações ao branch `main`.
7. **Limpeza:** Após o merge, você pode excluir o branch local e remoto para manter o repositório organizado.

```
# Volta para o main e atualiza
git checkout main
git pull origin main
# Deleta o branch local
git branch -d minha-nova-feature
```

4. Lidando com Conflitos de Merge

Conflitos são uma parte natural do trabalho colaborativo. Eles ocorrem quando o Git não consegue mesclar automaticamente as alterações de dois branches porque as mesmas linhas de um arquivo foram modificadas em ambos.

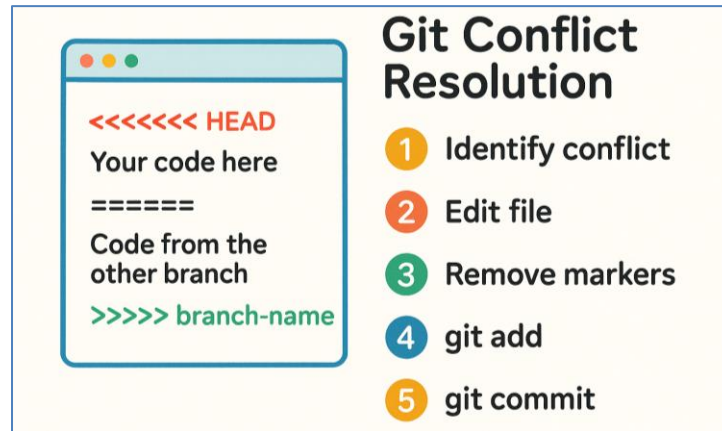


Figura 2: Ilustração do processo de resolução de conflitos no Git.

A figura 2 mostra os marcadores de conflito no arquivo e os passos necessários para resolver: identificar, editar, remover marcadores e finalizar com `git add` e `commit`.

4.1. Como Resolver um Conflito

1. **Identifique o Conflito:** O Git informará sobre o conflito durante um `merge` ou `pull`. O arquivo conflitante terá marcadores especiais:

```
<<<<<< HEAD
// Seu código (do branch atual)
=====
// Código vindo do outro branch
>>>>>> nome-do-outro-branch
```

2. **Abra o Arquivo:** Abra o arquivo em um editor de código.
3. **Decida o que Manter:** Edite o arquivo para remover os marcadores (<<<<<<, =====, >>>>>>) e deixe o código da forma que ele deve ficar. Você pode escolher manter o seu código, o código do outro branch, ou uma combinação de ambos.
4. **Adicione o Arquivo Resolvido:** Após editar o arquivo, salve-o e adicione-o ao stage para marcar o conflito como resolvido.

```
git add nome-do-arquivo-conflitante
```

5. **Complete o Merge:** Continue o processo de merge com um commit.

```
git commit
```

O Git geralmente cria uma mensagem de commit padrão para o merge, que você pode simplesmente confirmar.

5. Boas Práticas de Colaboração

- Commits Atômicos: Faça commits pequenos e focados em uma única alteração. Isso facilita a revisão e a identificação de bugs.
- Mensagens de Commit Claras: Escreva mensagens de commit descritivas. O padrão comum é ter um título curto (até 50 caracteres) e, se necessário, um corpo explicando o *porquê* da alteração.
- Mantenha seu Branch Atualizado: Antes de começar a trabalhar e antes de abrir um PR, sempre atualize seu branch `main` (`git pull origin main`) e, se necessário, mescle o `main` em seu branch de feature para resolver conflitos localmente.
- Pull Requests Descritivos: Escreva um bom título e uma descrição clara para seus Pull Requests, explicando o que foi feito e por quê. Se o PR resolve uma issue, mencione o número da issue.
- Revise o Código dos Outros: A revisão de código é uma via de mão dupla. Revise os PRs de seus colegas para aprender com o código deles e ajudar a manter a qualidade do projeto.

6. Exercícios Práticos

6.1. Exercício 1: Criando e Mesclando um Pull Request

- a) Crie um fork de um repositório público (você pode criar um repositório de teste em sua própria conta para isso).
- b) Clone o seu fork para a máquina local.
- c) Crie um novo branch chamado `minha-contribuicao`.
- d) Faça uma alteração simples em um arquivo (ex: corrija um erro de digitação no `README.md`).
- e) Faça o push do seu branch para o seu fork no GitHub.
- f) Abra um Pull Request do seu branch `minha-contribuicao` no seu fork para o branch `main` do repositório original.
- g) Como dono do repositório, faça o merge do seu próprio Pull Request.

6.2. Exercício 2: Simulando e Resolvendo um Conflito

- a) Peça a um colega para fazer um fork do seu repositório de teste.
- b) Ambos devem clonar seus respectivos forks.
- c) Você: No seu repositório local, no branch `main`, edite a primeira linha do `README.md` e faça o push.
- d) Seu colega: No repositório local dele, no branch `main`, edite a **mesma** primeira linha do `README.md` com um texto diferente e tente fazer o push. O push será rejeitado.
- e) Seu colega: Agora, ele deve fazer um `git pull` para baixar suas alterações. O Git irá acusar um conflito de merge.
- f) Seu colega: Peça para ele resolver o conflito, editar o arquivo, fazer o `add` e o `commit` do merge, e então fazer o `push` final.

7. Soluções dos exercícios práticos

7.1. Exercício 1: Criando e Mesclando um Pull Request

Este exercício tem como objetivo ensinar o fluxo completo de contribuição em projetos usando Pull Requests através da prática de: Criação de fork de repositório; Clonagem e configuração local; Criação de branches para contribuições; Submissão de Pull Request; Processo de revisão e merge. Antes de começar, verifique se os requisitos abaixo estão cumpridos:

- Conta no GitHub ativa
- Git instalado e configurado
- Editor de texto disponível
- Navegador web para interface do GitHub

Pull Request (PR) é uma funcionalidade do GitHub que permite:

- Propor alterações para um repositório
- Revisar código antes da integração
- Discutir mudanças com a equipe
- Manter qualidade do código
- Documentar histórico de contribuições

O fluxo conceitual do pull request é: 1.Repositório Original → 2.Fork → 3.Clone Local → 4.Branch → 5.Alterações → 6.Push → 7.Pull Request → 8.Merge

Verificação Inicial

```
# Verificar instalação do Git
git --version

Verificar configuração
git config --global user.name
git config --global user.email
```

Item A: Criar um Fork de Repositório Público

Passo 1: Criar Repositório de Teste (Preparação)

Opção A: Criar seu próprio repositório para teste

1. Acesse GitHub: <https://github.com>
2. Clique em "New repository"
3. Configure o repositório:
 - Repository name: projeto-teste-pr
 - Description: Repositório para praticar Pull Requests
 - Visibility: Public
 - Initialize with README: Marcar esta opção
4. Clique em "Create repository"

Opção B: Usar repositório existente

Se preferir, pode usar qualquer repositório público existente na sua conta.

Passo 2: Simular Fork (Usando Conta Secundária ou Colaborador)

Método 1: Conta secundária

1. Crie uma segunda conta no GitHub (ou use uma existente)
2. Acesse o repositório criado na conta principal
3. Clique em "Fork" no canto superior direito
4. Selecione a conta de destino para o fork
5. Aguarde a criação do fork

Método 2: Mesmo usuário (para fins didáticos)

1. Acesse seu repositório `projeto-teste-pr`
2. Anote a URL: `https://github.com/SEU_USUARIO/projeto-teste-pr`
3. Considere este como repositório "original"
4. Crie um segundo repositório chamado `projeto-teste-pr-fork`
5. Clone o original e push para o fork manualmente

Passo 3: Verificar Fork Criado

1. Acesse o fork na conta secundária ou repositório fork
2. Verifique se aparece "forked from USUARIO_ORIGINAL/projeto-teste-pr"
3. Anote as URLs:
 - Original: `https://github.com/USUARIO_ORIGINAL/projeto-teste-pr`
 - Fork: `https://github.com/USUARIO_FORK/projeto-teste-pr`

Resultado Esperado: Fork criado com sucesso

Item B: Clonar o Fork para Máquina Local

Passo 1: Copiar URL do Fork

1. Acesse o repositório fork no GitHub
2. Clique no botão "Code" (verde)
3. Copie a URL HTTPS do fork
 - Exemplo: `https://github.com/USUARIO_FORK/projeto-teste-pr.git`

Passo 2: Clonar Localmente

```
# Navegar para diretório de trabalho
cd ~/Documents
# ou
mkdir projetos-git && cd projetos-git
```

```
# Clonar o fork (não o repositório original)
git clone https://github.com/USUARIO_FORK/projeto-teste-pr.git

# Entrar no diretório clonado
cd projeto-teste-pr
```

Passo 3: Configurar Repositório Upstream

```
# Adicionar repositório original como upstream
git remote add upstream https://github.com/USUARIO_ORIGINAL/projeto-teste-pr.git

# Verificar configuração de remotos
git remote -v
```

Saída Esperada:

```
origin      https://github.com/USUARIO_FORK/projeto-teste-pr.git (fetch)
origin      https://github.com/USUARIO_FORK/projeto-teste-pr.git (push)
upstream    https://github.com/USUARIO_ORIGINAL/projeto-teste-pr.git (fetch)
upstream    https://github.com/USUARIO_ORIGINAL/projeto-teste-pr.git (push)
```

Passo 4: Sincronizar com Upstream

```
# Buscar alterações do repositório original
git fetch upstream

# Verificar branches disponíveis
git branch -a
```

Resultado Esperado: Fork clonado e configurado com upstream

Item C: Criar Branch "minha-contribuicao"

Passo 1: Verificar Estado Atual

```
# Verificar branch atual
git branch --show-current

# Verificar status
git status

# Garantir que está no main atualizado
git checkout main
git pull upstream main
```

Passo 2: Criar e Mudar para Novo Branch

```
# Criar branch minha-contribuicao baseado no main
git checkout -b minha-contribuicao

# Alternativa moderna
git switch -c minha-contribuicao
```


Passo 3: Verificar Criação do Branch

```
# Verificar branch atual
git branch --show-current

# Listar todos os branches
git branch -a
```

Saída Esperada:

```
minha-contribuicao
main
* minha-contribuicao
remotes/origin/main
remotes/upstream/main
```

Resultado Esperado: Branch "minha-contribuicao" criado e ativo

Item d: Fazer Alteração Simples no README.md

Passo 1: Verificar Conteúdo Atual

```
# Visualizar conteúdo do README.md
cat README.md

# Listar arquivos disponíveis
ls -la
```

Passo 2: Editar README.md

Opção 1: Editor de linha de comando

```
# Editar com nano
nano README.md

# Ou com vim
vim README.md
```

Opção 2: Editor gráfico

```
# Abrir com VS Code
code README.md

# Ou outro editor
gedit README.md # Linux
notepad README.md # Windows
```

Passo 3: Fazer Alterações Específicas

Exemplo de alterações para fazer:

Conteúdo Original:

```
# projeto-teste-pr
Repositório para praticar Pull Requests
```

Conteúdo Modificado:

```
# Projeto Teste PR

Repositório para praticar Pull Requests e colaboração em equipe.

## Objetivo

Este projeto foi criado para demonstrar o fluxo de trabalho com Pull Requests, incluindo:

- Fork de repositórios
- Criação de branches
- Submissão de contribuições
- Processo de revisão de código

## Como Contribuir

1. Faça um fork do projeto
2. Crie um branch para sua feature (`git checkout -b minha-contribuicao`)
3. Faça commit das suas alterações (`git commit -am 'Adiciona nova feature'`)
4. Faça push para o branch (`git push origin minha-contribuicao`)
5. Abra um Pull Request

## Autor

Exercício de Git e GitHub - Pull Requests

---

*Última atualização: [DATA_ATUAL]*
```

Passo 4: Salvar Alterações

1. Salve o arquivo (Ctrl+S ou comando do editor)
2. Feche o editor

Passo 5: Verificar Alterações

```
# Ver diferenças feitas
git diff
```

```
# Verificar status dos arquivos
git status
```

Saída Esperada do git status:

```
On branch minha-contribuicao
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes to working directory)
    modified:   README.md
no changes added to commit (use "git add" to track)
```

Resultado Esperado: README.md modificado com melhorias

Item E: Push do Branch para o Fork

Passo 1: Adicionar Alterações ao Stage

```
# Adicionar README.md modificado
git add README.md
```

```
# Verificar status após add
git status
```

Saída Esperada:

```
On branch minha-contribuicao
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   README.md
```

Passo 2: Fazer Commit

```
# Criar commit com mensagem descritiva
git commit -m "Melhora documentação do README com instruções de contribuição"
```

Saída Esperada:

```
[minha-contribuicao abc1234] Melhora documentação do README com instruções de
contribuição
1 file changed, 15 insertions(+), 2 deletions(-)
```

Passo 3: Push para o Fork

```
# Enviar branch para o fork (origin)
git push -u origin minha-contribuicao
```

Saída Esperada:

```
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 456 bytes | 456.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'minha-contribuicao' on GitHub by visiting:
remote:   https://github.com/USUARIO_FORK/projeto-teste-pr/pull/new/minha-
contribuicao
remote:
To https://github.com/USUARIO_FORK/projeto-teste-pr.git
 * [new branch]      minha-contribuicao -> minha-contribuicao
Branch 'minha-contribuicao' set up to track remote branch 'minha-contribuicao' from
'origin'.
```

Passo 4: Verificar no GitHub

1. Acesse o fork no GitHub
2. Verifique se o branch "minha-contribuicao" aparece
3. Observe a mensagem sobre criar Pull Request

Resultado Esperado: Branch enviado para o fork no GitHub

Item F: Abrir Pull Request

Passo 1: Acessar Interface de Pull Request

Método 1: Link direto do push

1. Copie o link fornecido na saída do git push
2. Cole no navegador

Método 2: Interface do GitHub

1. Acesse o fork no GitHub
2. Clique em "Compare & pull request" (banner amarelo)
3. Ou clique em "Pull requests" → "New pull request"

Passo 2: Configurar Pull Request

Configurações importantes:

1. Base repository: USUARIO_ORIGINAL/projeto-teste-pr
2. Base branch: main
3. Head repository: USUARIO_FORK/projeto-teste-pr
4. Compare branch: minha-contribuicao

Passo 3: Preencher Informações do PR

Título:

Melhora documentação do README com instruções de contribuição

Descrição:

```
## Descrição

Este PR melhora a documentação do projeto adicionando:

- Seção "Objetivo" explicando o propósito do repositório
- Seção "Como Contribuir" com passo a passo para novos contribuidores
- Formatação melhorada do README
- Informações sobre o autor

## Tipo de Mudança

- [x] Documentação [ ] Bug fix [ ] Nova feature [ ] Breaking change

## Checklist

- [x] Testei as alterações localmente
- [x] A documentação está clara e bem formatada
- [x] Segui as convenções do projeto

## Screenshots (se aplicável)

N/A - Alterações apenas em documentação

## Informações Adicionais

Este é um exercício de aprendizado sobre Pull Requests.
```

Passo 4: Criar Pull Request

1. Revise todas as informações
2. Verifique os arquivos alterados na aba "Files changed"
3. Clique em "Create pull request"

Passo 5: Verificar PR Criado

1. Anote o número do Pull Request (ex: #1)
2. Verifique se aparece na lista de PRs do repositório original
3. Confirme que o status é "Open"

Resultado Esperado: Pull Request criado e visível no repositório original

Item G: Fazer Merge do Pull Request

Passo 1: Acessar como Dono do Repositório

1. Faça login na conta que possui o repositório original
2. Acesse o repositório original
3. Clique em "Pull requests"
4. Clique no PR criado

Passo 2: Revisar o Pull Request

Verificações importantes:

1. Aba "Conversation":
 - Leia a descrição
 - Verifique se não há conflitos
 - Observe os checks automáticos
2. Aba "Files changed":
 - Revise as alterações linha por linha
 - Verifique se as mudanças fazem sentido
 - Confirme que não há código malicioso
3. Aba "Commits":
 - Verifique os commits incluídos
 - Confirme que as mensagens são adequadas

Passo 3: Aprovar o Pull Request (Opcional)

Se quiser deixar um comentário de aprovação:

1. Clique em "Review changes"
2. Adicione comentário: "LGTM! Ótima melhoria na documentação."
3. Selecione "Approve"
4. Clique em "Submit review"

Passo 4: Fazer Merge

Opções de merge disponíveis:

1. Create a merge commit: Preserva histórico completo
2. Squash and merge: Combina commits em um só
3. Rebase and merge: Reaplica commits sem merge commit

Para este exercício, use "Create a merge commit":

1. Clique em "Merge pull request"
2. Confirme a mensagem de merge (pode editar se necessário)
3. Clique em "Confirm merge"

Passo 5: Verificar Merge Concluído

1. Observe a mensagem "Pull request successfully merged and closed"
2. Verifique que o PR agora mostra status "Merged"
3. Acesse o branch main do repositório original
4. Confirme que as alterações estão presentes

Passo 6: Limpeza (Opcional)

1. Clique em "Delete branch" para remover o branch do fork
2. Confirme a exclusão

Resultado Esperado: Pull Request mesclado com sucesso no repositório original

Exercício Concluído!

Verificação Final Completa

Execute os comandos abaixo para confirmar que tudo está correto:

```
# No repositório local, verificar estado final
git checkout main
git pull upstream main
git log --oneline -5
git branch -a
```

Verificação no GitHub

1. Repositório Original:
 - Verifique se o README.md foi atualizado
 - Confirme que o PR aparece como "Merged"
 - Observe o histórico de commits
2. Fork:
 - Verifique se o branch foi criado
 - Confirme que o PR foi enviado

Conceitos Importantes Aprendidos

1. Fluxo de Contribuição Open Source

Fork → Clone → Branch → Code → Commit → Push → Pull Request → Review → Merge

2. Diferença entre Origin e Upstream

- Origin: Seu fork (onde você tem permissão de escrita)
- Upstream: Repositório original (onde você quer contribuir)

3. Tipos de Merge no GitHub

Tipo	Descrição	Quando Usar
Merge Commit	Preserva histórico completo	Projetos que valorizam histórico
Squash and Merge	Combina commits em um	Limpar histórico de feature
Rebase and Merge	Reaplica commits linearmente	Manter histórico linear

4. Boas Práticas para Pull Requests

- Título claro e descritivo
- Descrição detalhada do que foi alterado
- Commits organizados com mensagens claras
- Testes das alterações antes do PR
- Revisão própria antes de submeter

Comandos Git Utilizados – Resumo

Comando	Descrição	Exemplo
<code>git clone <url></code>	Clona repositório	<code>git clone https://github.com/user/repo.git</code>
<code>git remote add upstream <url></code>	Adiciona repositório upstream	<code>git remote add upstream https://github.com/original/repo.git</code>
<code>git checkout -b <branch></code>	Cria e muda para branch	<code>git checkout -b minha-contribuicao</code>
<code>git add <file></code>	Adiciona arquivo ao stage	<code>git add README.md</code>
<code>git commit -m "msg"</code>	Cria commit	<code>git commit -m "Melhora documentação"</code>
<code>git push -u origin <branch></code>	Envia branch para fork	<code>git push -u origin minha-contribuicao</code>
<code>git fetch upstream</code>	Busca alterações do original	<code>git fetch upstream</code>
<code>git pull upstream main</code>	Puxa alterações do main original	<code>git pull upstream main</code>

Possíveis Problemas e Soluções

• Problema 1: "No permission to push"

```
# Erro ao fazer push para repositório original
remote: Permission to USUARIO_ORIGINAL/repo.git denied
```

```
#Solução: Verificar se está fazendo push para o fork
git remote -v
git push origin minha-contribuicao # Não upstream!
```

- **Problema 2: "Branch already exists"**
Se o branch já existir
git branch -d minha-contribuicao # Deletar local
git push origin --delete minha-contribuicao # Deletar remoto
git checkout -b minha-contribuicao # Criar novamente
- **Problema 3: "Merge conflicts"**
Se houver conflitos no PR
1. Sincronizar fork com upstream
2. Fazer merge do main no seu branch
3. Resolver conflitos manualmente
4. Fazer push das resoluções
- **Problema 4: "Fork out of sync"**
Sincronizar fork com repositório original
git checkout main
git fetch upstream
git merge upstream/main
git push origin main

Estado Final do Repositório

Após completar o exercício, seu repositório terá esta estrutura:

Repositório Original:

```
|—— main branch (atualizado com merge)
|   |—— README.md (melhorado)
|   |—— Pull Request #1 (merged)
```

Fork:

```
|—— main branch
|—— minha-contribuicao branch
|—— Pull Request enviado
```

Dicas para Contribuições Reais

- Leia o CONTRIBUTING.md do projeto
- Siga as convenções de código estabelecidas
- Teste suas alterações completamente
- Seja respeitoso nas discussões
- Aceite feedback construtivo

7.2. Exercício 2: Simulando e Resolvendo um Conflito

Este exercício tem como objetivo ensinar como identificar, entender e resolver conflitos de merge através da prática de: Simulação de desenvolvimento colaborativo; Identificação de conflitos de merge; Resolução manual de conflitos; Finalização de merge com conflitos resolvidos. Antes de começar, verifique se os requisitos abaixo estão cumpridos:

- Exercício 1 concluído (repositório de teste criado)
- Duas pessoas ou duas contas GitHub
- Git configurado em ambas as máquinas
- Comunicação entre os participantes

Preparação Inicial

- Repositório base: Use o repositório criado no Exercício 1
- Participantes: Pessoa A (dono) e Pessoa B (colaborador)
- Sincronização: Ambos devem estar com repositórios atualizados

O que são Conflitos de Merge?

Conflitos de merge ocorrem quando:

- Duas pessoas editam a mesma linha de um arquivo
- Git não consegue decidir automaticamente qual versão manter
- Intervenção manual é necessária para resolver
- Colaboração requer comunicação e decisões conscientes

Cenário Típico de Conflito

```
Repositório Original:  
Linha 1: "# Projeto Original"  
Pessoa A edita:  
Linha 1: "# Projeto Melhorado - Versão A"  
Pessoa B edita (simultaneamente):  
Linha 1: "# Projeto Atualizado - Versão B"  
Resultado: CONFLITO!
```

Item A: Fork do Repositório pelo Colega

Passo 1: Identificar Participantes

Pessoa A (Dono do Repositório):

- Possui o repositório original: projeto-teste-pr
- Fará alterações primeiro
- Enviará alterações para o repositório

Pessoa B (Colaborador):

- Fará fork do repositório da Pessoa A
- Fará alterações conflitantes
- Resolverá os conflitos

Passo 2: Pessoa B - Criar Fork

1. Pessoa B acessa o repositório da Pessoa A
 - URL: `https://github.com/PESSOA_A/projeto-teste-pr`
2. Clica em "Fork" no canto superior direito
3. Seleciona sua conta como destino do fork
4. Aguarda a criação do fork

Passo 3: Verificar Fork Criado

Pessoa B verifica:

1. Acessa seu fork: `https://github.com/PESSOA_B/projeto-teste-pr`
2. Confirma que aparece "forked from PESSOA_A/projeto-teste-pr"
3. Anota as URLs para referência:
 - Original: `https://github.com/PESSOA_A/projeto-teste-pr`
 - Fork: `https://github.com/PESSOA_B/projeto-teste-pr`

Resultado Esperado: Fork criado com sucesso pela Pessoa B

Item B: Clonagem dos Respectivos Forks

Passo 1: Pessoa A - Clonar Repositório Original

```
# Pessoa A clona seu próprio repositório
cd ~/projetos-git
git clone https://github.com/PESSOA_A/projeto-teste-pr.git pessoa-a-repo
cd pessoa-a-repo

# Verificar configuração
git remote -v
git status
```

Saída Esperada para Pessoa A:

```
origin https://github.com/PESSOA_A/projeto-teste-pr.git (fetch)
origin https://github.com/PESSOA_A/projeto-teste-pr.git (push)
```

```
On branch main
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean
```

Passo 2: Pessoa B - Clonar Fork

```
# Pessoa B clona seu fork
cd ~/projetos-git
git clone https://github.com/PESSOA_B/projeto-teste-pr.git pessoa-b-repo
cd pessoa-b-repo

# Configurar upstream para o repositório original
git remote add upstream https://github.com/PESSOA_A/projeto-teste-pr.git

# Verificar configuração
git remote -v
```

Saída Esperada para Pessoa B:

```
origin      https://github.com/PESSOA_B/projeto-teste-pr.git (fetch)
origin      https://github.com/PESSOA_B/projeto-teste-pr.git (push)
upstream    https://github.com/PESSOA_A/projeto-teste-pr.git (fetch)
upstream    https://github.com/PESSOA_A/projeto-teste-pr.git (push)
```

Passo 3: Sincronização Inicial

Ambas as pessoas executam:

```
# Verificar conteúdo atual do README.md
cat README.md

# Verificar status
git status

# Confirmar que estão no branch main
git branch --show-current
```

Resultado Esperado: Ambos têm repositórios clonados e sincronizados

Item C: Pessoa A - Primeira Alteração

Passo 1: Verificar Estado Atual

```
# Pessoa A verifica estado
cd pessoa-a-repo
git status
git branch --show-current
```

Passo 2: Editar Primeira Linha do README.md

Conteúdo Original (exemplo):

```
# Projeto Teste PR
Repositório para praticar Pull Requests e colaboração em equipe.
```

Pessoa A edita para:

```
# Projeto Teste PR - Versão Melhorada por A
Repositório para praticar Pull Requests e colaboração em equipe.
```

Comandos para edição:

```
# Pessoa A edita o arquivo
nano README.md
# ou
code README.md

# Salvar e fechar o editor
```

Passo 3: Verificar Alterações

```
# Pessoa A verifica diferenças
git diff

# Verificar status
git status
```

Saída Esperada do git diff:

```
diff --git a/README.md b/README.md
index abc1234..def5678 100644
--- a/README.md
+++ b/README.md
@@ -1,4 +1,4 @@
-# Projeto Teste PR
+# Projeto Teste PR - Versão Melhorada por A
Repositório para praticar Pull Requests e colaboração em equipe.
```

Passo 4: Commit e Push

```
# Pessoa A adiciona alterações
git add README.md

# Fazer commit
git commit -m "Atualiza título do projeto - versão A"
# Fazer push
git push origin main
```

Saída Esperada:

```
[main abc1234] Atualiza título do projeto - versão A
1 file changed, 1 insertion(+), 1 deletion(-)
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Writing objects: 100% (3/3), 298 bytes | 298.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
To https://github.com/PESSOA_A/projeto-teste-pr.git
def5678..ghi9012 main -> main
```

Passo 5: Verificar no GitHub

Pessoa A verifica:

1. Acessa seu repositório no GitHub
2. Confirma que o README.md foi atualizado
3. Informa Pessoa B que pode prosseguir

Resultado Esperado: Alteração da Pessoa A enviada com sucesso

Item D: Pessoa B - Alteração Conflitante

Passo 1: Pessoa B Edita a Mesma Linha

IMPORTANTE: Pessoa B NÃO deve fazer pull antes desta etapa!

```
# Pessoa B verifica estado (deve estar desatualizado)
cd pessoa-b-repo
git status
```

Pessoa B edita a primeira linha para:

```
# Projeto Teste PR - Versão Atualizada por B
Repositório para praticar Pull Requests e colaboração em equipe.
```

Comandos para edição:

```
# Pessoa B edita o arquivo
nano README.md
# ou
code README.md

# Salvar e fechar o editor
```

Passo 2: Verificar Alterações Locais

```
# Pessoa B verifica diferenças
git diff

# Verificar status
git status
```

Passo 3: Commit Local

```
# Pessoa B adiciona alterações
git add README.md

# Fazer commit
git commit -m "Atualiza título do projeto - versão B"
```

Saída Esperada:

```
[main jkl3456] Atualiza título do projeto - versão B
1 file changed, 1 insertion(+), 1 deletion(-)
```

Passo 4: Tentar Push (Será Rejeitado)

```
# Pessoa B tenta fazer push
git push origin main
```

Saída Esperada (PUSH REJEITADO):

```
To https://github.com/PESSOA_B/projeto-teste-pr.git
! [rejected]          main -> main (fetch first)
error: failed to push some refs to 'https://github.com/PESSOA_B/projeto-teste-pr.git'
hint: Updates were rejected because the remote contains work that you do
hint: not have locally. This is usually caused by another repository pushing
hint: to the same ref. You may want to first integrate the remote changes
hint: (e.g., 'git pull ...') before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

Passo 5: Entender a Rejeição

Explicação do erro:

- O repositório remoto (fork da Pessoa B) está desatualizado
- O repositório original (Pessoa A) foi atualizado
- Git não permite push que sobrescreva alterações

Resultado Esperado: Push rejeitado conforme esperado

Item E: Pessoa B - Pull e Conflito de Merge

Passo 1: Sincronizar com Repositório Original

```
# Pessoa B busca alterações do repositório original
git fetch upstream

# Verificar diferenças
git log --oneline upstream/main..main
git log --oneline main..upstream/main
```

Passo 2: Tentar Merge (Gerará Conflito)

```
# Pessoa B tenta fazer merge do upstream
git pull upstream main
```

Saída Esperada (CONFLITO DE MERGE):

```
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 1), reused 3 (delta 1), pack-reused 0
Unpacking objects: 100% (3/3), 278 bytes | 278.00 KiB/s, done.
From https://github.com/PESSOA_A/projeto-teste-pr
 * branch          main          -> FETCH_HEAD
    def5678..ghi9012 main        -> upstream/main
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
```

Passo 3: Verificar Estado do Conflito

```
# Pessoa B verifica status após conflito
git status
```

Saída Esperada:

```
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)
Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   README.md
no changes added to commit (use "git add" to track)
```

Passo 4: Examinar Arquivo com Conflito

```
# Pessoa B visualiza o arquivo com marcadores de conflito
cat README.md
```

Conteúdo com Marcadores de Conflito:

```
<<<<<<< HEAD
# Projeto Teste PR - Versão Atualizada por B
=====
# Projeto Teste PR - Versão Melhorada por A
>>>>>>> ghi9012 (Atualiza título do projeto - versão A)
```

Repositório para praticar Pull Requests e colaboração em equipe.

Explicação dos Marcadores:

```
<<<<<<< HEAD
# Projeto Teste PR - Versão Atualizada por B
=====
# Projeto Teste PR - Versão Melhorada por A
>>>>>>> ghi9012 (Atualiza título do projeto - versão A)
Repositório para praticar Pull Requests e colaboração em equipe.
```

Resultado Esperado: Conflito de merge identificado e visualizado

Item F: Pessoa B - Resolução do Conflito

Passo 1: Decidir Resolução do Conflito

Opções de resolução:

1. Manter versão A: Aceitar alteração da Pessoa A
2. Manter versão B: Manter alteração da Pessoa B
3. Combinar ambas: Criar versão que incorpora ambas
4. Criar nova versão: Escrever algo completamente novo

Para este exercício, vamos combinar ambas as versões:

Passo 2: Editar Arquivo para Resolver Conflito

```
# Pessoa B edita o arquivo para resolver conflito
nano README.md
# ou
code README.md
```

Versão Resolvida (exemplo):

```
# Projeto Teste PR - Versão Colaborativa (A + B)
```

Repositório para praticar Pull Requests e colaboração em equipe.

```
## Histórico de Alterações
```

- Versão melhorada por A
- Versão atualizada por B
- Conflito resolvido colaborativamente

```
## Como Contribuir
```

1. Faça um fork do projeto
2. Crie um branch para sua feature
3. Faça commit das suas alterações
4. Resolva conflitos se necessário
5. Faça push para o branch
6. Abra um Pull Request

IMPORTANTE: Remover TODOS os marcadores de conflito:

```
- <<<<<< HEAD
- =====
- >>>>>> commit
```

Passo 3: Verificar Resolução

```
# Pessoa B verifica se não há mais marcadores de conflito
grep -n "<<<\|==\|>>>" README.md

# Se não houver saída, os marcadores foram removidos
# Visualizar arquivo final
cat README.md
```

Passo 4: Adicionar Arquivo Resolvido

```
# Pessoa B adiciona arquivo resolvido
git add README.md

# Verificar status após add
git status
```

Saída Esperada:

```
On branch main
All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)
```

```
Changes to be committed:
  modified:   README.md
```

Passo 5: Commit do Merge

```
# Pessoa B faz commit do merge
git commit -m "Resolve conflito de merge: combina versões A e B"

- Mantém melhorias da versão A
- Incorpora atualizações da versão B
- Adiciona seção de histórico de alterações
- Melhora instruções de contribuição"
```

Saída Esperada:

```
[main mno7890] Resolve conflito de merge: combina versões A e B
```


Passo 6: Verificar Estado Após Merge

```
# Pessoa B verifica status
git status
# Ver histórico
git log --oneline -5
# Ver diferenças com o original
git diff upstream/main
```

Saída Esperada do git status:

```
On branch main
Your branch is ahead of 'origin/main' by 2 commits.
  (use "git push" to publish your local commits)
nothing to commit, working tree clean
```

Passo 7: Push Final

```
# Pessoa B faz push das alterações resolvidas
git push origin main
```

Saída Esperada:

```
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 8 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 756 bytes | 756.00 KiB/s, done.
Total 6 (delta 2), reused 0 (delta 0), pack-reused 0
To https://github.com/PESSOA_B/projeto-teste-pr.git
def5678..mno7890  main -> main
```

Resultado Esperado: Conflito resolvido e alterações enviadas com sucesso

Exercício Concluído!

Verificação Final Completa

Pessoa A verifica:

```
cd pessoa-a-repo
git pull origin main
git log --oneline -5
cat README.md
```

Pessoa B verifica:

```
cd pessoa-b-repo
git status
git log --oneline -5
cat README.md
```

Verificação no GitHub

Ambas as pessoas verificam:

1. Repositório da Pessoa A: Deve ter apenas sua alteração original
2. Fork da Pessoa B: Deve ter a versão resolvida do conflito
3. Histórico de commits: Deve mostrar o merge commit

Conceitos Importantes Aprendidos

1. Anatomia de um Conflito de Merge

```
<<<<<<< HEAD
Sua versão local
=====
Versão remota
>>>>>>> commit_hash
```

2. Fluxo de Resolução de Conflitos

Conflito Detectado → Examinar Arquivo → Editar Manualmente → git add → git commit → git push

3. Tipos de Resolução

Tipo	Descrição	Quando Usar
Aceitar Local	Manter sua versão	Quando sua versão está correta
Aceitar Remoto	Manter versão remota	Quando versão remota é melhor
Combinar	Mesclar ambas as versões	Quando ambas têm valor
Reescrever	Criar versão completamente nova	Quando nenhuma versão serve

4. Prevenção de Conflitos

- Comunicação constante entre equipe
- Pull frequente do repositório principal
- Divisão clara de responsabilidades
- Branches específicos para features

Comandos Git Utilizados – Resumo

Comando	Descrição	Exemplo
git pull upstream main	Puxa alterações e pode gerar conflito	git pull upstream main
git status	Mostra arquivos em conflito	git status
git diff	Mostra diferenças incluindo conflitos	git diff
git add <arquivo>	Marca conflito como resolvido	git add README.md
git commit	Finaliza merge após resolução	git commit -m "Resolve conflito"
git merge --abort	Cancela merge em andamento	git merge --abort
grep "<<< == >>>" arquivo	Procura marcadores de conflito	grep "<<<" README.md

Possíveis Problemas e Soluções

- Problema 1: "Cannot push while merge is in progress"

```
# Se tentar push durante merge não resolvido
error: cannot push while merge is in progress
```

```
# Solução: Resolver conflito primeiro
git status # Ver arquivos em conflito
# Editar arquivos
git add arquivo_resolvido
git commit
git push
```

- Problema 2: "Merge markers still present"

```
# Se ainda houver marcadores no arquivo
error: Merge conflict markers detected
```

```
# Solução: Procurar e remover todos os marcadores
grep -n "<<\\|===\\|>>" arquivo.txt
# Editar arquivo para remover marcadores
```

- Problema 3: "Abortar merge acidentalmente"

```
# Se quiser cancelar merge em andamento
git merge --abort
```

```
# Volta ao estado anterior ao merge
# Perde progresso de resolução de conflitos
```

- Problema 4: "Conflito em múltiplos arquivos"

```
# Ver todos os arquivos em conflito
git status
```

```
# Resolver um por vez
git add arquivo1_resolvido
git add arquivo2_resolvido
# ... resolver todos
git commit
```

Estado Final do Repositório

Após completar o exercício, seu repositório terá esta estrutura:

Repositório Pessoa A:

```
|— main branch  
|   |  
|   |— README.md (versão A)
```

Fork Pessoa B:

```
|— main branch  
|   |  
|   |— README.md (versão resolvida A+B)  
|   |  
|   |— Histórico com merge commit
```

Aprendizados:

```
|— Identificação de conflitos  
|— Resolução manual  
|— Comunicação em equipe  
|— Fluxo colaborativo
```

8. Dicas para Projetos Reais

- Sempre teste código após resolver conflitos
- Documente decisões de resolução importantes
- Use branches para isolar desenvolvimento
- Comunique-se proativamente com a equipe
- Faça backup antes de resolver conflitos complexos

9. Ferramentas Auxiliares para Resolução

Editores com Suporte a Conflitos

- VS Code:
 - Destaca conflitos automaticamente
 - Botões para aceitar versões
 - Visualização lado a lado
- Vim/Neovim:
 - Plugin vim-fugitive para Git
 - Comandos :Gdiff para comparação
 - Navegação entre conflitos

Ferramentas Gráficas:

```
# Configurar ferramenta de merge  
git config --global merge.tool vimdiff  
# ou  
git config --global merge.tool meld  
  
# Usar ferramenta para resolver conflitos  
git mergetool---
```

10. Boas Práticas para Conflitos

Antes do Conflito (Prevenção)

1. **Sincronize frequentemente:**
2. **Comunique alterações:**
 - Informe equipe sobre arquivos sendo editados
 - Use branches específicos para features
 - Faça commits pequenos e frequentes
3. **Divida responsabilidades:**
 - Evite múltiplas pessoas no mesmo arquivo
 - Use arquivos separados quando possível

Durante o Conflito (Resolução)

1. **Entenda o contexto:**
 - Leia as duas versões completamente
 - Entenda o objetivo de cada alteração
 - Comunique-se com outros desenvolvedores
2. **Resolva conscientemente:**
 - Não aceite automaticamente uma versão
 - Teste a versão resolvida
 - Documente decisões importantes
3. **Teste após resolução:**
 - Verifique se código ainda funciona
 - Execute testes se disponíveis
 - Revise alterações antes do commit

11. Cenários Avançados de Conflitos

Conflito em Múltiplas Linhas

```
<<<<<<< HEAD
# Projeto Teste PR - Versão B
Descrição atualizada por B
Novas funcionalidades adicionadas
=====
# Projeto Teste PR - Versão A
Descrição melhorada por A
Correções de bugs implementadas
>>>>>>> commit_hash
```

Conflito em Arquivos Binários

```
# Git não pode mesclar arquivos binários automaticamente
# Deve escolher uma versão:

# Manter versão local
git checkout --ours arquivo_binario.jpg

# Manter versão remota
git checkout --theirs arquivo_binario.jpg

# Marcar como resolvido
git add arquivo_binario.jpg
```